LAB REPORT

Fundamentals of Operating Systems (OS) Assignment - 2

Name : Gauransh Pawar

Roll No : 2401201120

Course : BCA (AI C DS)

Section : A

---

1. Introduction

Deadlock is a critical problem in operating systems where a set of processes are unable to proceed because each process is waiting for resources held by other processes. This situation can halt system execution and reduce efficiency.

To prevent deadlocks, operating systems use various techniques, one of which is the Banker's Algorithm. It is a deadlock avoidance algorithm that ensures the system always remains in a safe state, meaning all processes can complete execution without leading to deadlock.

In this assignment, Banker's Algorithm has been implemented using Python to determine whether a system is in a safe or unsafe state and to identify the safe sequence of execution.

2. Objectives

- To understand the concept of deadlock and its impact
- To implement Banker's Algorithm using Python
- To calculate the Need Matrix
- To determine whether the system is in a safe state
- To find the safe sequence of process execution
- To analyze system behavior based on resource allocation

3. Tools and Technologies Used

- Programming Language: Python 3.x

- Code Editor: Visual Studio Code
- Platform: Windows
- Libraries Used: Built-in Python functions

4. System Input and Data Representation

4.1 Overview

The system is defined using:

- Number of processes
- Number of resource types
- Allocation Matrix
- Maximum Matrix
- Available Resources

These components represent the current state of resource allocation in the system.

4.2 Allocation Matrix

The Allocation Matrix shows the number of resources currently allocated to each process

```python
allocation = []
print("\nEnter Allocation Matrix:")

for i in range(n):
    row = list(map(int, input(f"P{i}: ").split()))
    allocation.append(row)


print("\nAllocation Matrix:")
print("-" * 30)
for i in range(n):
    print(f"P{i}: {allocation[i]}")
```

✓ 21.8s

```
Enter Allocation Matrix:

Allocation Matrix:
------------------------------
P0: [0, 1, 0]
P1: [2, 0, 0]
P2: [3, 0, 2]
P3: [2, 1, 1]
P4: [0, 0, 2]
```

4.3 Maximum Matrix

The Maximum Matrix defines the maximum demand of each process for each resource type.

```
maximum = []
print("\nEnter Maximum Matrix:")

for i in range(n):
    row = list(map(int, input(f"P{i}: ").split()))
    maximum.append(row)


print("\nMaximum Matrix:")
print("-" * 30)
for i in range(n):
    print(f"P{i}: {maximum[i]}")
```

✓ 16.0s

```
Enter Maximum Matrix:

Maximum Matrix:
------------------------------
P0: [7, 5, 3]
P1: [3, 2, 2]
P2: [9, 0, 2]
P3: [2, 2, 2]
P4: [4, 3, 3]
```

4.4 Available Resources

The Available vector represents the number of resources currently available in the system.

```
print("\nAvailable Resources:")
print("-" * 25)
for i, val in enumerate(available):
    print(f"R{i}: {val}")
```

✓ 0.0s

```
Available Resources:
-------------------------
R0: 3
R1: 3
R2: 2
```

5. Need Matrix Calculation

5.1 Formula

The Need Matrix represents the remaining resource requirement of each process and is calculated using:

$$\textbf{Need} = \textbf{Maximum} - \textbf{Allocation}$$

5.2 Implementation

Each element of the Need Matrix is calculated by subtracting the Allocation value from the Maximum value for the corresponding process and resource.

5.3 Output

The Need Matrix is displayed in a structured tabular format for clarity

```python
print("\nNeed Matrix:")
print("-" * 30)

for i in range(n):
    print(f"P{i}: {need[i]}")
```

✓ 0.0s

```
Need Matrix:
------------------------------
P0: [7, 4, 3]
P1: [1, 2, 2]
P2: [6, 0, 0]
P3: [0, 1, 1]
P4: [4, 3, 1]
```

6. Banker's Safety Algorithm

6.1 Working Principle

The Banker's Algorithm checks whether the system is in a safe state by simulating process execution based on available resources.

6.2 Steps Involved

1. **Initialize:**
    - Work = Available
    - Finish = False for all processes
2. **Find a process such that:**

- Need ≤ Work

3. **If found:**
    - Allocate resources to the process
    - Update Work
    - Mark process as finished
4. **Repeat until:**
    - All processes are completed OR
    - No further allocation is possible

6.3 Key Logic

The condition used to check whether a process can execute is:

- **Need ≤ Available resources**

This ensures that the process can complete without causing deadlock.

```python
finish = [False] * n
safe_sequence = []
work = available.copy()

while len(safe_sequence) < n:
    found = False

    for i in range(n):
        if not finish[i]:

            if all(need[i][j] <= work[j] for j in range(m)):

                print(f"\nProcess P{i} is executing...")

                for j in range(m):
                    work[j] += allocation[i][j]

                safe_sequence.append(i)
                finish[i] = True
                found = True

                print(f"Updated Available (Work): {work}")

    if not found:
        break
```

✓ 0.0s

```
Process P1 is executing...
Updated Available (Work): [5, 3, 2]

Process P3 is executing...
Updated Available (Work): [7, 4, 3]

Process P4 is executing...
Updated Available (Work): [7, 4, 5]

Process P0 is executing...
Updated Available (Work): [7, 5, 5]

Process P2 is executing...
Updated Available (Work): [10, 5, 7]
```

7. Safe Sequence Determination

7.1 Definition

A Safe Sequence is an order of process execution such that all processes can complete without leading to deadlock.

7.2 Implementation

During execution, processes that satisfy the safety condition are added to the safe sequence.

7.3 Output

The safe sequence is displayed in order of execution.

```
    if len(safe_sequence) == n:
        print("\nSafe Sequence:")
        print("-" * 30)
        print(" -> ".join([f"P{i}" for i in safe_sequence]))
✓ 0.0s

Safe Sequence:
------------------------------
P1 -> P3 -> P4 -> P0 -> P2
```

8. Result Analysis

8.1 System State

After executing the algorithm, the system is classified as:

- Safe State → All processes completed successfully
- Unsafe State → Deadlock may occur

8.2 Observations

- If a safe sequence exists, the system is stable
- If no sequence exists, processes may get stuck indefinitely
- Proper resource allocation is essential to avoid deadlocks

```python
if len(safe_sequence) == n:
    print("\nAnalysis:")
    print("-" * 30)
    print("✓ All processes were able to execute successfully.")
    print("✓ Resources were allocated and released safely.")
    print("✓ No deadlock situation occurred.")
    print("✓ A safe sequence exists, ensuring system stability.")
else:
    print("\nAnalysis:")
    print("-" * 30)
    print("✗ Not all processes could be executed.")
    print("✗ Some processes are waiting indefinitely.")
    print("✗ Deadlock condition may occur.")
    print("✗ No safe sequence exists.")
```

✓ 0.0s

```
Analysis:
------------------------------
✓ All processes were able to execute successfully.
✓ Resources were allocated and released safely.
✓ No deadlock situation occurred.
✓ A safe sequence exists, ensuring system stability.
```

G. Explanation of Results

In this implementation:

- The Need Matrix correctly represents remaining resource requirements
- The algorithm successfully checks each process for execution feasibility
- Resources are allocated and released dynamically
- A safe sequence is generated when possible

This confirms that the system behavior aligns with theoretical expectations of Banker's Algorithm.

10. Challenges Faced

- Understanding matrix representation of resources

- Implementing correct Need calculation
- Handling edge cases where no process can execute
- Ensuring proper updating of available resources
- Avoiding infinite loops in the safety algorithm

11. Learning Outcomes

- Gained understanding of deadlock and avoidance techniques
- Learned how Banker's Algorithm ensures system safety
- Improved matrix handling and logical reasoning skills
- Understood real-world importance of resource allocation
- Developed structured and modular coding practices

12. Conclusion

Banker's Algorithm is an effective method for avoiding deadlocks by ensuring that the system always operates in a safe state. Through this assignment, it is observed that proper resource allocation and dynamic checking of process requirements are essential for system stability.

The implementation successfully demonstrates how deadlock can be avoided and how a safe sequence can be determined for process execution.